

TX13

MainMenuView (Capa de Presentación UI):

Funcionalidad:

Renderiza la interfaz de entrada de la aplicación en SwiftUI dentro del `@MainActor`. Responsable de exponer el botón para iniciar la captura de ecuaciones.

Propiedades y métodos:

`body: Some View`: Define la jerarquía visual de la pantalla inicial conforme a las *Human Interface Guidelines*.

`onTapCameraButton()`: Método de acción que captura el gesto de toque (*tap*) sobre el botón de la cámara y notifica inmediatamente al `MainMenuViewModel`.

MainMenuViewModel (Capa de Lógica de Presentación)

Funcionalidad:

Mantiene el estado reactivo de la vista (marcado con `@Observable` bajo Swift 6) y coordina las reglas de negocio previas al flujo de captura de la imagen.

Valida las condiciones necesarias antes de abrir la cámara. Verifica si la app tiene permiso de video o si el sistema está listo; si falta algo, detiene el flujo y lo solicita, y si todo está en orden, autoriza el paso a la captura de la imagen.

Propiedades y métodos:

`isCameraAuthorized: Bool`: Propiedad de estado que refleja si la app cuenta con los permisos del sistema operativo para acceder a la cámara.

`handleCameraButtonTap()`: Método orquestador. Determina si se deben solicitar permisos mediante el servicio o si se debe ordenar al coordinador la transición hacia la vista de la cámara.

CameraPermissionsService (Capa de Servicios de Infraestructura)

Funcionalidad:

Encapsula la API del sistema nativo (**AVFoundation**) para gestionar la autorización del hardware de video.

Es el encargado exclusivo de gestionar los permisos de hardware con iOS. Le pregunta al sistema operativo si el usuario ya autorizó el uso de la cámara y, en caso de no haberlo hecho, dispara la alerta nativa de Apple para pedir la autorización.

Propiedades y métodos:

checkPermission() **AsyncStatus**: Consulta asíncrona del estado actual de autorización (**AVCaptureDevice.authorizationStatus(for: .video)**) sin bloquear el hilo principal.

requestAccess() **AsyncBool**: Dispara la solicitud nativa de permisos del sistema (**AVCaptureDevice.requestAccess(for: .video)**) suspendiendo la ejecución con **async/await** hasta la respuesta del usuario.

AppCoordinator (Capa de Navegación)

Funcionalidad:

Administra la pila de navegación global de la aplicación (**NavigationStack**) desvinculando la jerarquía de vistas de la lógica de enrutamiento.

Es el director de tráfico de la aplicación. Controla el cambio entre pantallas (por ejemplo, pasar del Menú Principal a la Cámara) y administra el historial de navegación para avanzar o regresar, evitando que las pantallas tengan que llamarse directamente entre sí.

Propiedades y métodos:

path: NavigationPath: Almacena el historial de rutas y estados de navegación consumidos por el contenedor **NavigationStack** de SwiftUI.

navigateToCamera(): Muta la propiedad **path** apilando el destino correspondiente al flujo de la cámara.

Flujo:

Captura del Evento: El usuario interactúa con la interfaz. `MainMenuView` invoca `onTapCameraButton()`, transfiriendo el control a `MainMenuViewModel.handleCameraButtonTap()`.

Evaluación de Permisos: El `MainMenuViewModel` delega en `CameraPermissionsService.checkPermission()` la consulta del estado del hardware. Si el estado es indeterminado, invoca `requestAccess()` suspendiendo el contexto de forma asíncrona.

Actualización de Estado: Al recibir un resultado positivo (`true`), la propiedad `isCameraAuthorized` se actualiza en el `MainMenuViewModel`.

Enrutamiento: El ViewModel llama a `AppCoordinator.navigateToCamera()`, el cual modifica el `NavigationPath` provocando que SwiftUI renderice la vista de captura de la cámara en el `MainThread`.

TX14

CameraView (Capa de Presentación SwiftUI)

Funcionalidad:

Renderiza la vista de la cámara según el estado del ViewModel. Está completamente desacoplada de `AVFoundation`.

Propiedades:

`viewModel: CameraViewModel` vinculado mediante `@State` u `@ObservedObject` / `@Bindable` para reaccionar a cambios de estado.

`body: Some View`: Declaración de la jerarquía visual de SwiftUI que commuta componentes según `viewModel.state` (ej. barra de carga, mensaje de permiso denegado o el *preview layer* de la cámara).

CameraViewModel (Lógica de Presentación)

Funcionalidad:

Actúa como máquina de estados y orquestador entre la UI y el servicio subyacente. Se ejecuta bajo `@MainActor` para garantizar mutaciones *thread-safe* en el hilo principal de renderizado.

Propiedades y métodos:

`state: CameraViewState`: Estado reactivo expuesto a SwiftUI (`.idle`, `.loadingPermission`, `.ready`, etc.).

`checkAndRequestPermission() async`: Evalúa la autorización mediante `CameraServiceProtocol`. Si es `.notDetermined`, invoca la solicitud del sistema; si es `.authorized`, ejecuta `configureSession()` y transiciona el estado a `.ready`.

`retryPermission()`: Reintenta la evaluación de estado o redirige al usuario a la configuración del sistema si el permiso fue denegado.

CameraServiceProtocol (Interfaz de Dominio / Contrato)

Funcionalidad:

Define el contrato para cualquier implementación del motor de la cámara.
Facilita la inyección de dependencias y el uso de *Mocks* en pruebas unitarias.

Métodos y firmas:

`checkPermission()` -> `CameraAuthorizationStatus`: Inquire el estado actual de los permisos del hardware.

`requestPermission()` `async` -> `Bool`: Dispara de forma suspendible la solicitud de permisos al sistema operativo.

`configureSession()` `async` `throws`: Inicializa el hardware de captura y prepara la sesión de video.

AVCameraService (Servicio Concreto de Infraestructura)

Funcionalidad:

Implementación nativa que encapsula la API de `AVFoundation`.
Administra la captura de fotogramas en hilos de fondo.

Propiedades:

`-captureSession: AVCaptureSession`: Instancia responsable de gestionar el flujo de datos desde la cámara física hasta los buffers de salida.

`-deviceInput: AVCaptureDeviceInput`: Representa el hardware de la cámara del iPhone configurado como entrada de la sesión.

CameraViewState y CameraAuthorizationStatus (Modelos de Estado)

CameraViewState (Enumeración): Define los estados posibles de la interfaz gráfica:

- `.idle`: Estado inicial antes de realizar verificaciones.
- `.loadingPermission`: Proceso asíncrono de verificación en ejecución.
- `.denied`: Acceso denegado por el usuario o por restricciones del sistema.
- `.error`: Falla técnica al inicializar el `AVCaptureDeviceInput` o la `AVCaptureSession`.
- `.ready`: Sesión configurada y lista para capturar fotogramas para el pipeline de preprocesamiento.

CameraAuthorizationStatus (Enumeración): Mapeo de los estados de `AVAuthorizationStatus` (`.notDetermined`, `.restricted`, `.denied`, `.authorized`).

Flujo:

1. Iniciación y Consulta de Estado (Pasos 1 al 6)

1. **Acción del Usuario:** El usuario toca el botón **"Tomar foto"** en la pantalla `CameraView`.
2. **Cambio de Estado Inicial:** La vista notifica al `CameraViewModel`. Este pasa su estado a `.loadingPermission` (lo que muestra un indicador de carga en la pantalla) e invoca la verificación de permisos en `AVCameraService`.
3. **Consulta al Sistema Operativo:** `AVCameraService` (ejecutado en un hilo secundario mediante un `actor` de Swift 6) consulta a la API nativa de iOS `AVCaptureDevice.authorizationStatus`.
4. **Respuesta del Sistema:** `AVFoundation` devuelve el estado actual de los permisos del sensor de video, el cual se clasifica en uno de tres caminos principales: `.notDetermined`, `.authorized` o `.denied`.

2. Escenario A: Es la primera vez que se pide el permiso (`.notDetermined`)

Si el usuario nunca antes ha abierto la cámara en la app:

1. **Solicitud de Alerta:** El `CameraViewModel` le ordena al servicio solicitar el acceso. `AVCameraService` invoca `AVCaptureDevice.requestAccess`.

2. **Interacción Nativa:** iOS interrumpe la app y muestra la alerta del sistema preguntando: "*¿Permites a ToLaTeX acceder a la cámara?*".

A.1. El Usuario Presiona "Sí" (Permiso Concedido)

- `AVFoundation` retorna `true` al servicio y este notifica al ViewModel que el acceso fue concedido (`.authorized`).
- El ViewModel solicita inmediatamente la configuración de la captura mediante `configureSession()`.
- **Carga Exitosa de la Sesión:** El servicio vincula la cámara trasera (`AVCaptureDeviceInput`) a la sesión (`AVCaptureSession`). Si la GPU y el hardware responden correctamente, el ViewModel cambia su estado a `.ready` y SwiftUI renderiza la previsualización en vivo (*Preview Layer*) para tomar la foto.
- **Fallo Técnico de Hardware:** Si la cámara física está ocupada o falla la asignación de buffers en memoria, el servicio lanza una excepción (`throws CameraError`). El ViewModel cambia el estado a `.error`, mostrando una pantalla con el botón "**Volver a cargar**", el cual ejecuta `retryPermission()` al tocarlo.

A.2. El Usuario Presiona "No" (Permiso Denegado)

- `AVFoundation` retorna `false`. El servicio notifica que el estado es `.denied`.
- El ViewModel cambia su estado a `.denied`.
- La pantalla `CameraView` muestra un mensaje explicativo informando que la cámara es indispensable para capturar las ecuaciones y ofrece una opción para volver al menú principal o ir a los Ajustes de iOS.

3. Escenario B: Error de API o Tiempo de Espera del Sistema (`Error`)

Si ocurre un fallo inesperado de comunicación entre la aplicación y la API de `AVFoundation` (por ejemplo, un *timeout* del sistema operativo):

- `AVCameraService` captura el fallo y lanza un `CameraError`.
- El ViewModel muta su estado a `.error`.
- SwiftUI renderiza la interfaz de fallo con el botón "**Volver a cargar**" para permitir al usuario reintentar la inicialización invocando `retryPermission()`.

4. Escenario C: El Permiso ya fue Denegado Previamente (.denied)

Si el usuario ya había rechazado el permiso en una sesión anterior:

- `AVCameraService` detecta inmediatamente el estado `.denied` desde el sistema sin volver a mostrar la alerta nativa.
- El ViewModel actualiza el estado a `.denied`.
- La vista SwiftUI muestra directamente la pantalla informativa indicando que la función está bloqueada y da la opción de volver al menú inicial o redirigir a los Ajustes de iOS.

TX15

CameraView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz gráfica del visor de la cámara en vivo, los controles de captura/reintento y la previsualización de la fotografía tomada.

Reacciona a los cambios de estado expuestos por el ViewModel para conmutar la pantalla entre el modo de encuadre y el modo de revisión.

Propiedades:

body: Some View: Declaración de la jerarquía visual adaptativa que alterna entre el feed de la cámara en tiempo real (*Preview Layer*) y la previsualización de la imagen capturada según la propiedad de estado del ViewModel.

CameraViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Actúa como la máquina de estados y orquestador reactivo (etiquetado con **@MainActor** y **@Observable** en Swift 6). Coordina la toma fotográfica con el servicio de hardware, gestiona la previsualización inmediata para el usuario y delega la imagen capturada al servicio de preprocesamiento antes de continuar en el pipeline.

Propiedades y métodos:

sessionState: CameraSessionState: Estado reactivo que representa el momento actual de la sesión (ej. **.configuring**, **.running**, **.previewingCapturedPhoto**, **.processing**).

capturedPhotoPreview: CGImage?: Propiedad de estado que almacena la representación rápida de la fotografía en memoria para renderizar la previsualización en SwiftUI.

startCameraSession() async: Inicia de forma asíncrona la sesión de la cámara al cargarse la vista.

takePhoto() async: Envía la orden de disparo a **CameraService**, recibe el objeto **AVCapturePhoto** y genera la imagen ligera para la previsualización.

`confirmPhoto() async`: Envía la foto capturada a `ImagePreprocessingService` para transformarla en `CVPixelBuffer` y notifica que la imagen está lista para la siguiente etapa del pipeline.

`retakePhoto()`: Descarta `capturedPhotoPreview`, reanuda la transmisión del sensor y restablece el estado para permitir una nueva toma.

CameraService (Capa de Servicio Concreto de Infraestructura):

Funcionalidad:

Implementado como un `Actor` de Swift 6 para garantizar la seguridad de hilos (*thread-safety*) en el acceso a la API de `AVFoundation`. Encapsula el control directo de la `AVCaptureSession` y el disparo físico del sensor óptico.

Propiedades y métodos:

-`captureSession`: `AVCaptureSession`: Instancia responsable de gestionar el flujo continuo de datos de video desde la cámara física.

-`photoOutput`: `AVCapturePhotoOutput`: Componente de hardware encargado de procesar y entregar la captura de alta resolución en formato `AVCapturePhoto`.

`configureSession() async throws`: Inicializa y vincula las entradas/salidas de hardware en hilos secundarios sin congelar la interfaz del usuario.

`capturePhotoData() async throws -> AVCapturePhoto`: Asigna los ajustes de captura y ejecuta el disparo físico suspendiendo la ejecución hasta recibir el objeto de la foto.

`stopSession()`: Detiene la transmisión de la `AVCaptureSession` para liberar recursos de hardware y GPU.

ImagePreprocessingService (Capa de Servicio de Procesamiento de Imagen):

Funcionalidad:

Actor de infraestructura responsable de convertir la foto cruda generada por el sensor (`AVCapturePhoto`) en un formato matricial estandarizado (`CVPixelBuffer`), preparándola para los algoritmos de `UIImage/CoreImage` y el modelo UniMERNet.

Métodos y firmas:

`prepareForPipeline(photo: AVCapturePhoto) async throws ->`

`CVPixelBuffer`: Extrae los datos fotográficos sin comprimir, ajusta la orientación nativa y genera una instancia de `CVPixelBuffer` optimizada en memoria compartida para el procesamiento de IA.

Flujo:

1. Inicialización y Arranque de Cámara

Acción del Usuario / Vista: Al cargarse `CameraView`, se invoca `CameraViewModel.startCameraSession()`.

Configuración de Hardware: El ViewModel llama asíncronamente a `CameraService.configureSession()`. El Actor configura la `AVCaptureSession` y la salida `AVCapturePhotoOutput` en un hilo secundario y, al finalizar con éxito, el ViewModel pasa la sesión al estado activo (`.running`), mostrando el video en vivo en SwiftUI.

2. Captura Fotográfica

Disparo de Captura: El usuario presiona el botón "Tomar foto" y la vista invoca `CameraViewModel.takePhoto()`.

Procesamiento de Hardware: El ViewModel ordena a `CameraService.capturePhotoData()` ejecutar la captura. El servicio activa el sensor óptico mediante `AVCapturePhotoOutput` y devuelve un objeto `AVCapturePhoto` de alta resolución.

3. Muestra de Previsualización

Generación de Previsualización: `CameraViewModel` extrae un buffer ligero `CGImage` del objeto `AVCapturePhoto` recibido y lo asigna a `capturedPhotoPreview`.

Cambio de Estado Visual: El ViewModel conmuta su `sessionState` a `.previewingCapturedPhoto`. SwiftUI reacciona deteniendo visualmente el visor y mostrando la imagen fija con los botones "Repetir" y "Usar Foto".

4. Decisión del Usuario (Repetir o Confirmar)

Opción A (Repetir Foto):

El usuario toca "Repetir". Se invoca `CameraViewModel.retakePhoto()`.

El ViewModel limpia `capturedPhotoPreview`, reanuda la transmisión del visor mediante `CameraService` y restablece `sessionState` a `.running` para capturar una nueva toma.

Opción B (Confirmar Foto):

El usuario toca "Usar Foto". Se invoca `CameraViewModel.confirmPhoto()`.

El ViewModel cambia el estado a `.processing` y envía el objeto `AVCapturePhoto` al actor

`ImagePreprocessingService.prepareForPipeline(photo:)`.

Conversión de Memoria y Salida al Pipeline

Transformación de Imagen: `ImagePreprocessingService` procesa la foto cruda en segundo plano y devuelve una instancia optimizada de `CVPixelBuffer`.

Avanzar en el Flujo: El ViewModel recibe el `CVPixelBuffer` procesado y delega la navegación al coordinador global de la app para continuar hacia la pantalla de ajuste o delimitación de ecuaciones.

TX17

CropView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz interactiva para el encuadre y corrección de perspectiva de la hoja o ecuación. Visualiza la imagen base junto con la superposición del polígono interactivo de 4 esquinas que el usuario puede ajustar manualmente mediante gestos táctiles.

Propiedades y métodos:

-viewModel: CropViewModel: Instancia del ViewModel vinculada para reaccionar a los cambios de estado de la vista y delegar acciones del usuario.

body: Some View: Declaración de la jerarquía visual de SwiftUI que presenta la imagen de entrada, los estados de carga o error y la capa interactiva de recorte.

-renderPolygonOverlay(): View: Método privado que renderiza las líneas del cuadrilátero y los tiradores táctiles (*handles*) sobre los vértices del **PerspectivePolygon** para permitir el ajuste manual sobre la imagen.

CropViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo (etiquetado con **@MainActor** y **@Observable** en Swift 6) para la pantalla de recorte. Orquesta la detección automática de contornos, mantiene la posición de los vértices ajustables y coordina la ejecución de la transformación de homografía en segundo plano sin congelar la UI.

Propiedades y métodos:

inputImage: CImage: Imagen de entrada a procesar mediante Core Image.

detectedPolygon: PerspectivePolygon?: Estado reactivo que almacena las coordenadas de los cuatro vértices del cuadrilátero activo (detectado automáticamente o ajustado por el usuario).

isProcessing: Bool: Indicador de estado para deshabilitar controles o mostrar visores de carga durante tareas asíncronas de análisis o procesamiento.

errorMessage: String?: Almacena el mensaje descriptivo en caso de fallos en la detección de bordes o en la transformación geométrica.

detectDocumentBounds() async: Invoca al servicio **DocumentDetectionService** para localizar automáticamente los bordes del documento o ecuación en la imagen.

updateCorner(index: Int, newLocation: CGPoint): Actualiza la posición de uno de los 4 vértices del polígono al ser arrastrado manualmente en la interfaz.

confirmAndCrop() async -> UIImage?: Dispara la aplicación de la homografía cuadrilátera a través de **ImageProcessingService** y retorna la imagen rectificada.

PerspectivePolygon (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (**struct**) de dominio que representa la geometría de un cuadrilátero en un plano bidimensional. Encapsula las coordenadas de los cuatro vértices de la hoja de papel y provee validaciones geométricas.

Propiedades y métodos:

topLeft: CGPoint: Coordenada de la esquina superior izquierda.

topRight: CGPoint: Coordenada de la esquina superior derecha.

bottomLeft: CGPoint: Coordenada de la esquina inferior izquierda.

bottomRight: CGPoint: Coordenada de la esquina inferior derecha.

isValid: Bool: Propiedad calculada que verifica si el cuadrilátero formado es convexo y no posee aristas cruzadas ni vértices superpuestos.

`toNormalizedRect()` -> `CGRect`: Calcula y retorna el rectángulo delimitador (*bounding box*) ajustado al espacio de coordenadas normalizado.

DocumentDetectionService (Capa de Servicios Core Image / Infraestructura):

Funcionalidad:

Actor de Swift 6 dedicado al análisis visual y detección analítica de contornos en segundo plano. Aplica umbralización y algoritmos de detección de bordes utilizando `CoreImage` para ubicar automáticamente la región de la ecuación o documento.

Métodos y firmas:

`detectDocument(in image: CImage) async throws -> PerspectivePolygon`: Método principal que analiza la `CImage` y retorna los 4 vértices del cuadrilátero detectado.

`-applyThresholdAndFilter(to image: CImage) -> CImage`: Método privado que aplica filtros de contraste y detección de bordes para aislar el papel del fondo.

`-extractCorners(from image: CImage) throws -> PerspectivePolygon`: Método privado que analiza las regiones de contraste máximo y extrae las coordenadas límite del cuadrilátero.

ImageProcessingService (Capa de Servicios Core Image / Infraestructura):

Funcionalidad:

Actor de Swift 6 encargado de ejecutar transformaciones geométricas sobre la imagen. Aplica corrección de perspectiva mediante una matriz de homografía proyectiva con los filtros nativos de `CoreImage`.

Métodos y firmas:

`applyHomography(to image: CImage, polygon: PerspectivePolygon) async throws -> CImage`: Recibe la imagen original junto con las coordenadas del cuadrilátero y aplica la corrección de perspectiva (`CIPerspectiveCorrection`), entregando una `CImage` aplanada y perfectamente rectangular.

Flujo:

1. Carga e Invocación de Detección Automática

Acción del Usuario / Vista: Al cargarse `CropView`, se invoca `CropViewModel.detectDocumentBounds()`.

Procesamiento Asíncrono: El ViewModel marca `isProcessing = true` y delega la tarea en segundo plano al actor `DocumentDetectionService.detectDocument(in:)`.

Análisis de Contornos: `DocumentDetectionService` procesa la `CImage` aplicando umbralización mediante `applyThresholdAndFilter` y calcula los vértices del documento con `extractCorners`, retornando una estructura `PerspectivePolygon`.

Renderizado Inicial: `CropViewModel` asigna el polígono a `detectedPolygon`, cambia `isProcessing = false` y `CropView` renderiza la superposición del cuadrilátero interactivo mediante `renderPolygonOverlay()`.

2. Edición Manual de Vértices (Ajuste Fino)

Interacción Táctil: El usuario arrastra una de las esquinas en la pantalla. `CropView` captura el gesto e invoca `CropViewModel.updateCorner(index: newLocation:)`.

Actualización de Estado: `CropViewModel` actualiza la coordenada correspondiente en `detectedPolygon`, valida la coherencia geométrica mediante `PerspectivePolygon.isValid` y refresca la UI en el `@MainActor`.

3. Confirmación y Corrección de Perspectiva (Homografía)

Confirmación del Usuario: El usuario presiona el botón "Confirmar / Recortar". Se invoca `CropViewModel.confirmAndCrop()`.

Ejecución en Actor: El ViewModel pasa a `isProcessing = true` e invoca asíncronamente a `ImageProcessingService.applyHomography(to: polygon:)`.

Transformación Geométrica: `ImageProcessingService` aplica la matriz de homografía con `CoreImage` proyectando los 4 puntos seleccionados hacia un plano rectangular regular sin distorsiones de perspectiva.

4. Entregable y Transición de Estado

Resultado Exitoso: El servicio retorna la nueva **CImage** rectificada.

CropViewModel finaliza la tarea asíncrona y entrega la imagen procesada al coordinador de la app para continuar hacia la etapa de delimitación de ROIs o preparación de tensores.

Manejo de Excepciones: Si no se encuentran contornos válidos o falla la transformación geométrica, los servicios lanzan un error, **CropViewModel** actualiza **errorMessage** y **CropView** muestra la alerta correspondiente en la interfaz.

TX18

PerspectiveCorrectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz de previsualización de la imagen con la perspectiva rectificadas. Proporciona estados visuales reactivos para indicar el procesamiento matricial en la GPU, despliega el resultado aplanado final y expone las opciones para confirmar la imagen o navegar hacia el ajuste manual si el resultado no es óptimo.

Propiedades y métodos:

-viewModel: PerspectiveTransformViewModel: Instancia del ViewModel vinculada para reaccionar a las mutaciones de estado y canalizar eventos de usuario.

body: Some View: Jerarquía visual adaptativa en SwiftUI que commuta entre el indicador de carga, la vista de error o la representación de la imagen procesada.

-renderLoadingState(): View: Método privado que renderiza la vista de progreso asíncrono mientras Core Image ejecuta la matriz de homografía.

-renderPreviewState(image: Image): View: Método privado que presenta la imagen corregida lista para la validación visual del usuario.

PerspectiveTransformViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con **@MainActor** y **@Observable** bajo Swift 6. Orquesta la invocación asíncrona del servicio de homografía en segundo plano, controla los estados de interfaz y gestiona los flujos de contingencia (reintento de transformación o retorno a la vista de ajuste manual).

Propiedades y métodos:

inputImage: UIImage: Representación en memoria de la fotografía original a procesar.

polygon: PerspectivePolygon: Geometría que contiene los 4 vértices del cuadrilátero delimitador.

correctedImage: CImage?: Almacena la **CImage** resultante tras aplicar la corrección de perspectiva.

isProcessing: Bool: Estado reactivo que bloquea interacciones en la UI mientras la GPU realiza el cálculo de la homografía.

errorMessage: String?: Registra mensajes de error en caso de invaildez geométrica o fallos en los filtros de Core Image.

applyTransformation() async: Ejecuta la corrección de perspectiva invocando al actor **PerspectiveCorrectionService**.

retryTransformation() async: Reintenta el procesamiento asíncrono en caso de fallos de inicialización en el pipeline.

navigateToManualAdjustment(): Ordena al coordinador de navegación regresar a **CropView** para modificar manualmente los vértices del polígono.

confirmAndProceed() async: Valida la **CImage** rectificada y transiciona hacia la etapa de delimitación de ROIs.

PerspectivePolygon (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (**struct**) de la capa de dominio que define la geometría cuadrilátera de la hoja o ecuación. Provee la lógica de conversión de coordenadas desde el espacio cartesiano de la UI al formato de vectores requerido por Core Image.

Propiedades y métodos:

topLeft: CGPoint: Coordenada de la esquina superior izquierda.

topRight: CGPoint: Coordenada de la esquina superior derecha.

bottomLeft: CGPoint: Coordenada de la esquina inferior izquierda.

`bottomRight: CGPoint`: Coordenada de la esquina inferior derecha.

`toCoreImageDictionary(imageSize: CGSize) -> [String: CIVector]`:
Convierte los 4 puntos de la estructura a un diccionario con claves nativas (`inputTopLeft`, `inputTopRight`, etc.) conteniendo instancias de `CIVector` adaptadas a la escala real de la imagen.

PerspectiveCorrectionService (Capa de Servicios Core Image / Infraestructura):

Funcionalidad:

Servicio encapsulado en un `Actor` de Swift 6 para asegurar la ejecución *thread-safe* en hilos secundarios. Invoca el motor de `CoreImage` con aceleración de hardware por Metal/GPU para aplicar la transformación de homografía proyectiva (`CIPerspectiveCorrection`).

Métodos y firmas:

`correctPerspective(in image: CImage, using polygon: PerspectivePolygon) async throws -> CImage`: Método público principal que valida la geometría y ejecuta la corrección de perspectiva sobre la imagen.

`-validatePolygonGeometry(_ polygon: PerspectivePolygon) throws`:
Método privado que comprueba que el polígono sea convexo y no tenga aristas cruzadas antes de someterlo al filtro de GPU.

`-applyCIPerspectiveCorrection(to image: CImage, vectors: [String: CIVector]) throws -> CImage`: Método privado que construye e invoca el filtro nativo `CIPerspectiveCorrection` de `CoreImage`, retornando la `CImage` resultante.

Flujo:

1. Invocación de Homografía Proyectiva

Acción del Usuario / Vista: Al cargarse `PerspectiveCorrectionView`, se dispara automáticamente `PerspectiveTransformViewModel.applyTransformation()`.

Procesamiento Asíncrono: El ViewModel asigna `isProcessing = true` y transfiere el control en segundo plano al actor

`PerspectiveCorrectionService.correctPerspective(in:using:)`.

2. Validación Geométrica y Aplicación de Filtro en GPU

Validación de Vértices: `PerspectiveCorrectionService` invoca

`-validatePolygonGeometry` para verificar la convexidad de las coordenadas de `PerspectivePolygon`.

Construcción de Vectores: La estructura `PerspectivePolygon` convierte sus puntos a `[String: CIVector]` mediante `toCoreImageDictionary`.

Ejecución en Core Image: El servicio ejecuta `-applyCIPerspectiveCorrection` utilizando el filtro nativo `CIPerspectiveCorrection` acelerado por GPU/Metal, retornando la nueva `CImage` aplanada sin deformaciones de perspectiva.

3. Renderizado de Previsualización y Decisión de Usuario

Actualización de Estado: `PerspectiveTransformViewModel` recibe la `CImage` corregida, la asigna a `correctedImage`, desactiva `isProcessing` y SwiftUI renderiza el resultado mediante `renderPreviewState()`.

Opción A (Confirmar Resultado):

El usuario valida la previsualización y presiona "Confirmar". Se ejecuta

`PerspectiveTransformViewModel.confirmAndProceed()`, enviando la `CImage` rectificada al coordinador para avanzar hacia la pantalla de selección de ecuaciones (ROIs).

Opción B (Regresar a Ajuste Manual):

Si la perspectiva corregida presenta recortes indebidos o imprecisiones, el usuario selecciona "Ajustar Manualmente". Se ejecuta

`PerspectiveTransformViewModel.navigateToManualAdjustment()`, lo que devuelve la navegación a `CropView` conservando los puntos anteriores para reajustarlos.

Gestión de Contingencias y Errores

Fallo de Procesamiento: Si la validación de geometría falla o el motor de Core Image reporta un error de asignación de memoria GPU, el servicio lanza una excepción (`throws`).

Reintento: `PerspectiveTransformViewModel` captura la excepción, asigna el mensaje descriptivo a `errorMessage` y `PerspectiveCorrectionView` muestra la pantalla de error permitiendo reintentar mediante `retryTransformation()`.

TX19

ImageAdjustmentView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz de ajuste de exposición y contraste fotométrico tras la captura y rectificación de la hoja. Muestra la previsualización en tiempo real de la ecuación procesada y expone la paleta interactiva de filtros (**dark**, **neutral**, **light**) para optimizar el trazo del texto matemático antes de la segmentación.

Propiedades y métodos:

-viewModel: ImageAdjustmentViewModel: Instancia del ViewModel observada en la vista para reaccionar a los cambios de estado y canalizar interacciones del usuario.

body: Some View: Jerarquía visual adaptativa en SwiftUI que despliega la imagen filtrada en pantalla y los controles interactivos del selector de filtros.

-renderFilterPalette(): View: Método privado que dibuja los botones/iconos de la paleta de niveles de ajuste para permitir la conmutación rápida entre configuraciones de brillo.

ImageAdjustmentViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con **@MainActor** y **@Observable** en Swift 6. Retiene la imagen original inmutable en memoria, administra la opción de filtrado activa y orquesta la ejecución del procesamiento fotométrico en el actor de infraestructura sin bloquear los 120 Hz de la interfaz gráfica.

Propiedades y métodos:

selectedFilter: FilterLevel: Estado reactivo que indica el nivel de filtro seleccionado actualmente por el usuario.

displayedImage: CGImage?: Representación actual de la imagen filtrada lista para ser renderizada por SwiftUI.

-**baseImage: CGImage**: Referencia inmutable de la foto original para recalcular los filtros desde cero en cada ajuste, evitando la degradación acumulativa de píxeles.

-**processingService: ImageProcessingServiceProtocol**: Interfaz inyectada del servicio de procesamiento de imágenes.

applyFilter(level: FilterLevel) async: Evalúa el nivel de filtro seleccionado y delega asincrónicamente el cálculo fotométrico al actor en segundo plano.

confirmSelection(): Vinda la **CGImage** procesada final y la transfiere al coordinador de navegación para avanzar a la vista de delimitación de regiones de interés (ROIs).

ImageProcessingService (Capa de Servicios Core Image / Infraestructura):

Funcionalidad:

Servicio encapsulado como un **Actor** de Swift 6 para garantizar la seguridad de hilos (*thread-safety*) en operaciones en segundo plano. Utiliza un contexto de Core Image (**CImageContext**) respaldado por Metal para ejecutar transformaciones fotométricas aceleradas por hardware sobre la matriz de la imagen.

Propiedades y métodos:

-**ciContext: CImageContext**: Instancia reutilizable de Core Image optimizada para ejecutar operaciones de renderizado directamente sobre la GPU/Metal.

applyColorAdjustment(image: CGImage, brightness: Float, contrast: Float) async throws -> CGImage: Método asíncrono que recibe los parámetros fotométricos, aplica los filtros de ajuste (**CIColorControls**) y retorna la nueva **CGImage** procesada en GPU sin duplicaciones innecesarias de memoria.

FilterLevel (Modelo de Estado / Enumeración):

Funcionalidad:

Enumeración de dominio que define las preconfiguraciones de ajuste de exposición y contraste diseñadas para resaltar el trazo de la tinta y eliminar sombras de fondo en la hoja.

Casos y propiedades:

case dark: Ajuste diseñado para imágenes sobreexpuestas o con trazos tenues (incrementa el contraste y reduce el brillo).

case neutral: Mantiene las condiciones fotométricas originales de la imagen sin alteraciones.

case light: Ajuste diseñado para eliminar sombras de fondo o baja iluminación (aumenta el brillo y resalta los oscuros).

brightnessValue: Float: Propiedad calculada que entrega el valor numérico flotante de brillo asociado a cada caso para alimentar el filtro de Core Image.

Flujo:

1. Carga Inicial y Presentación de Imagen

Acción del Usuario / Vista: Al cargarse `ImageAdjustmentView`, `ImageAdjustmentViewModel` asigna `baseImage` a `displayedImage` e inicializa `selectedFilter` en el caso `.neutral`.

Renderizado Inicial: SwiftUI presenta la imagen original en pantalla junto a la paleta de filtros generada por `-renderFilterPalette()`.

2. Selección de Filtro y Procesamiento Asíncrono en GPU

Interacción del Usuario: El usuario selecciona una opción en la paleta de filtros (ej. `.light`).

Invocación Asíncrona: `ImageAdjustmentView` invoca `ImageAdjustmentViewModel.applyFilter(level: .light)`. El ViewModel actualiza la propiedad `selectedFilter` e invoca asíncronamente al actor `ImageProcessingService.applyColorAdjustment(image:baseImage, brightness:level.brightnessValue, contrast:...)`.

Ejecución en Core Image / Metal: `ImageProcessingService` toma la `CGImage` base inmutable, configura el filtro `CIColorControls` con el valor de `brightnessValue`

correspondiente y procesa el resultado en la GPU utilizando `ciContext`. El actor retorna la nueva `CGImage` sin bloquear el hilo principal.

3. Actualización de Interfaz y Confirmación

Renderizado de Resultado: `ImageAdjustmentViewModel` recibe la nueva `CGImage` procesada, la asigna a `displayedImage` y SwiftUI actualiza la vista en el `@MainActor`.

Confirmación de Filtro: El usuario valida la claridad del trazo matemático y presiona "Confirmar". Se ejecuta `ImageAdjustmentViewModel.confirmSelection()`, enviando la imagen filtrada al `AppNavigationCoordinator` para continuar hacia la pantalla de selección de ecuaciones (ROIs).

TX20

EquationSelectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz del lienzo interactivo sobre la imagen filtrada. Permite al usuario trazar, ajustar y eliminar manualmente rectángulos delimitadores (ROIs - Regiones de Interés) para seleccionar una o múltiples ecuaciones matemáticas presentes en la misma hoja.

Propiedades y métodos:

-viewModel: EquationSelectionViewModel: Instancia del ViewModel observada en SwiftUI para reaccionar a mutaciones en la colección de ROIs y canalizar eventos de gestos táctiles.

body: Some View: Jerarquía visual adaptativa que muestra la imagen base y superpone las capas del lienzo de dibujo y las cajas delimitadoras activas.

-renderCanvasOverlay(): View: Método privado que dibuja la capa de captura de gestos táctiles (DragGesture) sobre la imagen para registrar el trazo inicial del usuario.

-renderInteractiveBoundingBoxes(): View: Método privado que renderiza visualmente la lista de EquationROI activas, dibujando sus bordes, identificadores y controles táctiles de edición o borrado.

EquationSelectionViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con @MainActor y @Observable bajo Swift 6. Mantiene la lista ordenable de regiones delimitadas (EquationROI), gestiona la ROI activa en tiempo real mientras el usuario arrastra el dedo y orquesta la extracción asíncrona de los sub-recortes de imagen.

Propiedades y métodos:

equationROIs: List: Colección reactiva (pila/array) que almacena todas las regiones de interés confirmadas por el usuario.

activeROI: EquationROI?: Estado reactivo que representa la región que se encuentra en proceso de trazado o modificación interactiva en el instante actual.

baseFilteredImage: CGImage: Referencia inmutable a la imagen procesada y rectificada que sirve como matriz base para el cálculo de coordenadas.

-cropService: ROICropServiceProtocol: Interfaz inyectada del servicio de recorte de imágenes.

addROI(normalizedRect: CGRect): Agrega una nueva región de interés a la lista **equationROIs** transformando la selección a coordenadas normalizadas $[0.0, 1.0]$.

updateActiveROI(normalizedRect: CGRect): Actualiza reactivamente las dimensiones de **activeROI** durante el evento de arrastre táctil en la pantalla.

undoLastROI(): Elimina el último rectángulo agregado de la pila **equationROIs**.

confirmSelection() async: Invoca el recorte matricial de las sub-imágenes mediante **ROICropService** y canaliza los resultados hacia la etapa de revisión e inferencia.

ROICropService (Capa de Servicios de Infraestructura / Actor):

Funcionalidad:

Servicio encapsulado como un **Actor** de Swift 6 para ejecutar operaciones de corte matricial en hilos secundarios de fondo. Mapea las coordenadas normalizadas a píxeles reales de la imagen y extrae las sub-imágenes **CGImage** sin bloquear la interfaz gráfica.

Métodos y firmas:

cropAndNormalizeRegions(image: CGImage, rois: List) async throws -> List: Recibe la **CGImage** base y la colección de ROIs, calcula la geometría

exacta en píxeles mediante `toPixelCoordinates` y retorna la lista de sub-imágenes recortadas en memoria.

EquationROI (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (`struct`) de dominio que representa una Región de Interés dentro de la imagen. Almacena su identificador único para la renderización en SwiftUI y sus coordenadas relativas normalizadas.

Propiedades y métodos:

`id: UUID`: Identificador único de la ROI para diferenciarla en colecciones declarativas (`ForEach`).

`normalizedBounds: CGRect`: Rectángulo que almacena la posición y dimensiones en espacio normalizado `CGRect` \in [0.0, 1.0]\$, independiente de la resolución del dispositivo.

`toPixelCoordinates(imageSize: CGSize) -> CGRect`: Método de cálculo geométrico que convierte las coordenadas normalizadas a píxeles absolutos de la matriz de la `CGImage` origen.

Flujo:

1. Carga del Lienzo e Inicialización

Acción del Usuario / Vista: Al cargarse `EquationSelectionView`, SwiftUI despliega la `baseFilteredImage` y activa la capa de gestos `-renderCanvasOverlay()`.

2. Trazado y Edición Interactiva de ROIs

Gesto de Arrastre: El usuario toca la pantalla y arrastra el dedo sobre una ecuación. La vista captura las coordenadas relativas y llama continuamente a `EquationSelectionViewModel.updateActiveROI(normalizedRect:)`.

Finalización del Gesto: Al levantar el dedo, se invoca `addROI(normalizedRect:)`. El ViewModel empaqueta la selección en una estructura `EquationROI` con su `UUID` y la apila en `equationROIs`.

Redibujado Declarativo: `EquationSelectionView` reacciona a la mutación renderizando el nuevo cuadro delimitador mediante `-renderInteractiveBoundingBoxes()`.

Deshacer Selección: Si el usuario desea remover la última selección, presiona el botón "Deshacer", ejecutando `undoLastROI()`.

3. Confirmación y Recorte Matricial Asíncrono

Confirmación: El usuario valida las regiones trazadas y presiona "Confirmar Selección". Se ejecuta `EquationSelectionViewModel.confirmSelection()`.

Procesamiento en Actor: El ViewModel invoca asíncronamente al actor `ROICropService.cropAndNormalizeRegions(image:baseFilteredImage, rois:equationROIs)`.

Mapeo a Píxeles: `ROICropService` itera sobre la lista, ejecuta `EquationROI.toPixelCoordinates(imageSize:)` para obtener la resolución real y realiza la extracción de los sub-buffers de la `CImage`.

4. Entrega de Sub-Imágenes al Pipeline

Resultado: `ROICropService` retorna la lista de sub-imágenes recortadas (`List<CImage>`). El ViewModel las recibe y notifica al `AppNavigationCoordinator` para avanzar hacia la previsualización de recortes y la posterior generación de tensores para UniMERNet.

TX21

EquationSelectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza los componentes de alerta visual, modales de confirmación y banners informativos cuando no existen regiones de interés (ROIs) seleccionadas o cuando el usuario solicita una purga masiva de selecciones sobre la imagen de la ecuación.

Propiedades y métodos:

-viewModel: EquationSelectionViewModel: Instancia del ViewModel observada en SwiftUI para conmutar dinámicamente las capas de la interfaz y reaccionar a la gestión de borrado.

body: Some View: Jerarquía visual adaptativa que integra el lienzo de selección junto con las superposiciones de confirmación modal y banners de advertencia de estado vacío.

-renderEmptyStateBanner(): View: Método privado que dibuja una barra o banner flotante informativo cuando `equationROIs` se encuentra vacía, alertando al usuario que debe delimitar al menos una ecuación antes de avanzar.

-renderConfirmationOverlay(): View: Método privado que renderiza un diálogo o modal de alerta pidiendo confirmación explícita para prevenir el borrado accidental de todas las ROIs trazadas.

EquationSelectionViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con `@MainActor` y `@Observable` bajo Swift 6. Administra la validez del lote de regiones trazadas, controla las banderas booleanas de la interfaz gráfica para evitar acciones destructivas accidentales y valida las reglas de negocio previas al envío a inferencia.

Propiedades y métodos:

equationROIs: **List:** Colección reactiva de regiones de interés seleccionadas sobre la imagen.

showClearConfirmation: **Bool:** Estado reactivo que controla la visibilidad del modal de alerta para el borrado completo de ROIs.

showEmptyStateBanner: **Bool:** Estado reactivo que indica si debe desplegarse la advertencia de falta de selecciones activas.

requestClearAll(): Método que activa **showClearConfirmation = true** al presionar el botón "Limpiar todo", suspendiendo la acción hasta confirmación explícita del usuario.

confirmClearAll(): Elimina la totalidad de los elementos de **equationROIs**, restablece **showClearConfirmation = false** y activa **showEmptyStateBanner = true**.

cancelClearAll(): Oculta la alerta de confirmación restableciendo **showClearConfirmation = false** sin alterar el contenido de **equationROIs**.

dismissEmptyStateBanner(): Oculta la notificación de estado vacío cuando el usuario reanuda el trazado de una nueva ROI.

EquationROI (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (**struct**) de dominio que representa las coordenadas de una región delimitada sobre la matriz de la foto.

Propiedades y métodos:

id: **UUID:** Identificador único inmutable para garantizar un rendimiento $O(1)$ en diffing declarativo dentro de colecciones de SwiftUI.

normalizedBounds: **CGRect:** Límites geométricos del área seleccionada expresados en espacio de coordenadas normalizadas $\text{CGRect} \in [0.0, 1.0]$.

Flujo:

1. Solicitud de Borrado Masivo (Clear All)

Acción del Usuario: El usuario cuenta con varias ROIs trazadas en pantalla y presiona el botón "Limpiar todo".

Intercepción: `EquationSelectionView` captura el evento e invoca `EquationSelectionViewModel.requestClearAll()`.

Actualización de UI: El ViewModel asigna `showClearConfirmation = true` y SwiftUI renderiza la overlay de advertencia destructiva mediante `-renderConfirmationOverlay()`.

2. Confirmación o Cancelación del Borrado

Opción A (Confirmar Borrado):

El usuario toca "Confirmar" en la alerta modal. Se ejecuta `EquationSelectionViewModel.confirmClearAll()`.

El ViewModel vacía la colección `equationROIs`, cierra el modal (`showClearConfirmation = false`) y conmuta `showEmptyStateBanner = true` para reflejar visualmente la ausencia de regiones.

Opción B (Cancelar Borrado):

El usuario toca "Cancelar". Se ejecuta `EquationSelectionViewModel.cancelClearAll()`.

El ViewModel restablece `showClearConfirmation = false` dejando intacta la lista `equationROIs` y el lienzo en pantalla.

Gestión de Estado Vacío (Empty State) e Intento de Avance

Intento Inválido de Navegación: Si el usuario presiona "Continuar" sin ROIs activas en la lista, el ViewModel detecta `equationROIs.isEmpty` y fuerza `showEmptyStateBanner = true`.

Notificación Visual: `EquationSelectionView` reacciona renderizando el banner de advertencia mediante `-renderEmptyStateBanner()`.

Reanudación de Trazado: En cuanto el usuario vuelve a tocar la pantalla para trazar una nueva región, el ViewModel invoca

`dismissEmptyStateBanner()`, ocultando la advertencia e iniciando el registro del nuevo polígono.

TX24

EquationSelectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz gráfica para la gestión interactiva y eliminación individual de Regiones de Interés (ROIs). Dibuja cada caja delimitadora sobre el lienzo de la imagen y posiciona dinámicamente un botón contextual de eliminación en cada región, presentando un diálogo modal de confirmación antes de aplicar cambios destructivos en la lista de ecuaciones.

Propiedades y métodos:

-viewModel: EquationSelectionViewModel: Instancia del ViewModel observada en SwiftUI para conmutar componentes visuales y transmitir la orden de borrado individual.

body: Some View: Jerarquía visual que superpone las cajas delimitadoras sobre la imagen base, ancla los controles táctiles de eliminación y gestiona el modal emergente de confirmación.

-renderROIBoundingBox(roi: EquationROI): View: Método privado que dibuja el contorno visual de una región delimitada específica a partir de sus dimensiones.

-renderDeleteButton(for roi: EquationROI): View: Método privado que genera el botón interactivo de borrado (ícono "X") y lo ancla en la esquina superior derecha del rectángulo mediante `topRightAnchor`.

-renderDeleteConfirmationModal(): View: Método privado que renderiza la vista modal emergente para solicitar confirmación explícita antes de eliminar la ROI seleccionada.

EquationSelectionViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con `@MainActor` y `@Observable` bajo Swift 6. Controla el flujo de selección y eliminación individual de regiones,

mantiene la referencia temporal de la ROI marcada para borrado y realiza mutaciones deterministas sobre la colección `equationROIs`.

Propiedades y métodos:

`equationROIs: List`: Colección reactiva de regiones de interés activas en la pantalla.

`roiPendingDeletion: EquationROI?`: Estado reactivo que retiene la instancia de la ROI que el usuario ha solicitado eliminar, antes de ser confirmada.

`showDeleteConfirmation: Bool`: Estado reactivo que controla la visibilidad del modal de confirmación de eliminación puntual.

`requestDeleteROI(id: UUID)`: Localiza la ROI dentro de `equationROIs` mediante su identificador único, la asigna a `roiPendingDeletion` y conmuta `showDeleteConfirmation = true`.

`confirmDeleteROI()`: Remueve la ROI almacenada en `roiPendingDeletion` de la colección `equationROIs`, restablece `roiPendingDeletion = nil` y cierra el modal asignando `showDeleteConfirmation = false`.

`cancelDeleteROI()`: Limpia la referencia en `roiPendingDeletion` y oculta el modal modalizando `showDeleteConfirmation = false` sin modificar la colección.

EquationROI (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (`struct`) de dominio que abstrae las dimensiones de una región de interés y provee métodos geométricos de ayuda para la ubicación de controles en la interfaz de usuario.

Propiedades y métodos:

`id: UUID`: Identificador único inmutable que garantiza búsquedas y eliminaciones eficientes en tiempo $O(1)$ en colecciones declarativas de SwiftUI.

normalizedBounds: CGRect: Rectángulo de la región delimitada expresado en coordenadas normalizadas `CGRect` \in [0.0, 1.0]\$.

topRightAnchor(in rect: CGRect) -> CGPoint: Método de cálculo geométrico que retorna la coordenada exacta de la esquina superior derecha del rectángulo dentro del espacio de coordenadas de la vista para anclar el botón de eliminación.

Flujo:

1. Solicitud de Eliminación Puntual

Acción del Usuario: El usuario toca el botón de eliminación (ícono "X") sobre una ROI específica renderizada en el lienzo con `-renderDeleteButton(for:)`.

Intercepción de Evento: `EquationSelectionView` captura el toque e invoca `EquationSelectionViewModel.requestDeleteROI(id: roi.id)`.

Actualización de Estado: El ViewModel identifica la estructura por su `UUID`, la asigna a `roiPendingDeletion` y conmuta `showDeleteConfirmation = true`. SwiftUI renderiza la ventana modal mediante `-renderDeleteConfirmationModal()`.

2. Confirmación o Cancelación de la Eliminación

Opción A (Confirmar Eliminación):

El usuario toca "Eliminar" en la alerta modal. Se ejecuta `EquationSelectionViewModel.confirmDeleteROI()`.

El ViewModel elimina la región de `equationROIs`, restablece `roiPendingDeletion = nil` y ajusta `showDeleteConfirmation = false`. SwiftUI redibuja el lienzo removiendo la ROI y su botón asociado en tiempo real.

Opción B (Cancelar Eliminación):

El usuario toca "Cancelar". Se ejecuta `EquationSelectionViewModel.cancelDeleteROI()`.

El ViewModel limpia `roiPendingDeletion` y asigna `showDeleteConfirmation = false`, dejando la colección `equationROIs` e interfaz intactas.

TX27

EquationSelectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz final de confirmación de lote antes de enviar las ecuaciones seleccionadas al pipeline de reconocimiento. Presenta una vista modal de confirmación previa a la inferencia y despliega un estado de carga global (`isProcessing`) que inhabilita las interacciones táctiles mientras los actores de fondo procesan los recortes y ejecutan el modelo en el Apple Neural Engine (ANE).

Propiedades y métodos:

`-viewModel: EquationSelectionViewModel`: Instancia del ViewModel observada en SwiftUI para reaccionar al estado de procesamiento y controlar la presentación modal.

`body: Some View`: Jerarquía visual adaptativa que muestra el lienzo de selección, el modal de confirmación y el overlay de carga durante la inferencia de la IA.

`-renderConfirmationModal(): View`: Método privado que renderiza la vista emergente modal resumiendo la cantidad de ecuaciones a procesar y solicitando la confirmación del usuario para iniciar la inferencia.

EquationSelectionViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con `@MainActor` y `@Observable` en Swift 6. Coordina la ejecución del pipeline completo de visión por computadora e inteligencia artificial: solicita el recorte de las regiones, la generación de tensores ajustados a 384×1152 px, delega la inferencia autoregresiva al actor de UniMERNet y transfiere los resultados al coordinador de navegación.

Propiedades y métodos:

`equationROIs: List`: Colección reactiva de las regiones de interés trazadas por el usuario.

showConfirmationModal: Bool: Estado reactivo que activa la visibilidad del modal emergente de confirmación previa.

isProcessing: Bool: Estado reactivo que bloquea la UI e indica la ejecución de recortes e inferencia en el ANE.

requestConfirmation(): Asigna **showConfirmationModal = true** al presionar el botón "Procesar Ecuaciones".

cancelConfirmation(): Oculta el modal asignando **showConfirmationModal = false**.

executeInferencePipeline() async: Método asíncrono principal. Desactiva el modal, establece **isProcessing = true**, solicita los tensores **CVPixelBuffer** a **ROICropService**, ejecuta la traducción a LaTeX con **UniMERNetInferenceService** y canaliza la salida hacia **AppNavigationCoordinator**.

ROICropService (Capa de Servicios / Infraestructura):

Funcionalidad:

Servicio encapsulado como un **Actor** de Swift 6 para procesar operaciones de visión en segundo plano. Mapea las coordenadas de cada **EquationROI**, realiza el corte matricial sobre la **CGImage** original y normaliza las dimensiones al tamaño exacto de entrada del modelo (384×1152 px) usando el framework **UIImage (Accelerate)**, generando instancias **CVPixelBuffer** listas para VRAM/ANE.

Métodos y firmas:

cropAndPrepareTensors(image: CGImage, rois: List) async throws -> List:
Corta sub-imágenes basadas en las ROIs, aplica escalado y padding cinemático a 384×1152 px y retorna la lista de buffers matriciales en formato **CVPixelBuffer**.

UniMERNetInferenceService (Capa de Servicios CoreML / IA):

Funcionalidad:

Servicio encapsulado como `Actor` en Swift 6 encargado de administrar la instancia del modelo UniMERNet compilado en CoreML (`.mlpackage`) cuantizado en FP16/INT8. Ejecuta la inferencia de reconocimiento de texto matemático optimizada para el Apple Neural Engine (ANE) y realiza la decodificación de tokens a sintaxis LaTeX.

Propiedades y métodos:

`-coreMLModel: MLModel`: Instancia interna inmutable de CoreML cargada en memoria.

`translateToLaTeX(pixelBuffer: CVPixelBuffer) async throws -> String`:

Método asíncrono que inyecta el `CVPixelBuffer` de 384 × 1152 px al ANE, ejecuta el pase autoregresivo del decodificador y retorna la cadena con el código LaTeX generado.

AppNavigationCoordinator (Capa de Arquitectura / Navegación):

Funcionalidad:

Objeto de coordinación de rutas centralizado. Administra la pila de navegación declarativa (`NavigationPath`) para desacoplar a los ViewModels de las transiciones de pantalla en SwiftUI.

Propiedades y métodos:

`path: NavigationPath`: Pila de navegación manipulada por la vista principal mediante `NavigationStack`.

`navigateToResults(results: List)`: Inserta en la pila de navegación el destino de la pantalla de resultados transicionando la lista de entidades `LaTeXResult`.

Flujo:

1. Confirmación de Lote por el Usuario
Acción del Usuario: El usuario presiona el botón "Procesar Ecuaciones".

Intercepción: `EquationSelectionView` llama a `EquationSelectionViewModel.requestConfirmation()`.

UI: ViewModel asigna `showConfirmationModal = true` y SwiftUI dibuja `-renderConfirmationModal()`.

2. Inicio del Pipeline e Inhabilitación de UI

Confirmación: El usuario presiona "Iniciar Reconocimiento" en el modal. Se ejecuta `executeInferencePipeline()`.

Estado: ViewModel oculta el modal (`showConfirmationModal = false`), activa `isProcessing = true` y la vista muestra la pantalla de carga.

3. Preparación de Tensores en `vImage` (Actor `ROICropService`)

Recorte Asíncrono: ViewModel llama a `ROICropService.cropAndPrepareTensors(image:baseImage, rois:equationROIs)`.

Procesamiento Matricial: El actor calcula las coordenadas reales, recorta cada región, aplica el reescalado y padding mediante `vImage` a la matriz de 384×1152 px y retorna una colección de `CVPixelBuffer`.

4. Inferencia en Apple Neural Engine (Actor `UniMERNetInferenceService`)

Decodificación IA: Por cada `CVPixelBuffer`, ViewModel invoca `UniMERNetInferenceService.translateToLaTeX(pixelBuffer:)`.

Ejecución en ANE: El actor pasa el tensor a CoreML, ejecuta la inferencia acelerada por ANE y retorna la cadena de texto con el resultado en sintaxis LaTeX.

5. Transición a Resultados

Navegación: ViewModel empaqueta los strings resultantes en instancias `LaTeXResult`, desactiva `isProcessing = false` e invoca `AppNavigationCoordinator.navigateToResults(results:)`.

Cambio de Pantalla: El coordinador empuja la nueva ruta a `path` y SwiftUI presenta la vista de edición y exportación de LaTeX.

TX28

CropReviewView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz de revisión final de las sub-imágenes recortadas antes de iniciar la traducción a LaTeX. Muestra los recortes ordenados en una cuadrícula/lista con su índice de secuencia y expone la barra de acciones superior para confirmar la inferencia o retornar a la pantalla de selección si se requiere capturar más ecuaciones.

Propiedades y métodos:

-viewModel: CropReviewViewModel: Instancia del ViewModel observada para reaccionar a cambios en la lista de recortes, estados de procesamiento y canalizar las decisiones del usuario.

body: Some View: Jerarquía visual adaptativa que conmuta entre la vista de revisión de recortes, el visor de carga durante la inferencia y la cabecera de acciones principales.

-renderCroppedItemsGrid(): View: Método privado que renderiza la cuadrícula ordenada de los objetos `CroppedEquationItem`, mostrando la imagen recortada junto a su número ordinal.

-renderActionHeader(): View: Método privado que dibuja la barra superior con los botones de acción "Volver a tomar más ecuaciones" y "Confirmar y ver ecuación en LaTeX".

CropReviewViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con `@MainActor` y `@Observable` en Swift 6. Administra la colección ordenada de recortes (`CroppedEquationItem`), controla el indicador de progreso `isProcessing` e interactúa con el actor de IA y el coordinador de navegación para dirigir el flujo global.

Propiedades y métodos:

croppedItems: List: Colección reactiva ordenada de las sub-imágenes recortadas listas para inferencia.

isProcessing: Bool: Estado reactivo que deshabilita la interfaz y activa la pantalla de carga durante la ejecución del pipeline en la GPU/ANE.

-inferenceService: UniMERNetInferenceServiceProtocol: Interfaz inyectada del servicio de inferencia autorregresiva de IA.

-coordinator: AppNavigationCoordinator: Referencia al coordinador global para gestionar los saltos en la pila de navegación.

confirmAndTranslate() async: Método asíncrono que activa **isProcessing = true**, envía el lote de recortes a **UniMERNetInferenceService** para la traducción en paralelo y canaliza los resultados traducidos hacia **AppNavigationCoordinator**.

returnToSelection(): Invoca a **AppNavigationCoordinator.popToSelectionView()** para regresar a la pantalla de delimitación manteniendo el historial de ROIs intacto.

CroppedEquationItem (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (**struct**) de la capa de dominio que empaqueta cada sub-imagen recortada junto con sus metadatos de secuencia y la ROI de origen.

Propiedades y métodos:

id: UUID: Identificador único de la entidad para el renderizado eficiente en colecciones de SwiftUI.

orderIndex: Int: Número ordinal que representa la posición de la ecuación dentro de la secuencia trazada por el usuario.

croppedImage: CGImage: Buffer de la imagen recortada y ajustada en memoria.

originalROI: EquationROI: Referencia a la región de interés geográfica de la cual proviene el recorte.

UniMERNetInferenceService (Capa de Servicios CoreML / IA):

Funcionalidad:

Servicio encapsulado como un **Actor** de Swift 6 para asegurar operaciones thread-safe en el Apple Neural Engine (ANE). Ejecuta el procesamiento por lote de las sub-imágenes, escalándolas y convirtiéndolas a tensores **CVPixelBuffer** para obtener la cadena en código LaTeX mediante el modelo UniMERNet.

Propiedades y métodos:

-coreMLModel: MLModel: Instancia compilada en memoria del modelo **.mlpackage** optimizado en FP16/INT8.

translateBatch(items: List) async throws -> List: Método asíncrono que procesa concurrentemente la lista de recortes, ejecuta el pase autorregresivo en el ANE y devuelve la lista ordenada de resultados traducidos a LaTeX.

AppNavigationCoordinator (Capa de Arquitectura / Navegación):

Funcionalidad:

Administrador de rutas centralizado que controla la pila **NavigationPath** de SwiftUI para gestionar la navegación entre vistas sin acoplamiento.

Propiedades y métodos:

path: NavigationPath: Pila declarativa de navegación consumida por el contenedor raíz.

popToSelectionView(): Retrocede en el historial de navegación retornando a **EquationSelectionView** sin destruir las selecciones previamente realizadas.

navigateToResults(results: List): Inyecta la ruta hacia la vista final de chat de resultados (**LaTeXChatResultsView**) adjuntando el lote de traducciones.

Flujo:

1. Presentación de Recortes y Carga de Vista

Acción del Usuario / Vista: Al navegar hacia `CropReviewView`, SwiftUI renderiza la lista ordenada de recortes procesados (`croppedItems`) mediante `-renderCroppedItemsGrid()` y presenta la cabecera de acciones `-renderActionHeader()`.

2. Opción A: Deseo de Tomar Más Ecuaciones

Interacción: El usuario evalúa las imágenes recortadas y decide agregar más fórmulas de la misma hoja presionando "Volver a tomar más ecuaciones".

Navegación: `CropReviewView` invoca `CropReviewViewModel.returnToSelection()`, la cual llama a `AppNavigationCoordinator.popToSelectionView()`. La app desapila la vista actual y regresa a `EquationSelectionView` conservando la imagen base y los polígonos trazados previamente.

3. Opción B: Confirmación e Inferencia en Lote (CoreML / ANE)

Confirmación: El usuario valida la lista de recortes y presiona "Confirmar y ver ecuación en LaTeX".

Inicio de Inferencia: Se ejecuta `CropReviewViewModel.confirmAndTranslate()`. El ViewModel conmuta `isProcessing = true`, inhabilitando los controles táctiles de la UI.

Procesamiento en Actor: El ViewModel invoca asíncronamente a `UniMERNetInferenceService.translateBatch(items: croppedItems)`. En el actor de IA, cada `CGImage` se normaliza a matriz de 384×1152 px (`CVPixelBuffer`), se inyecta en CoreML y se procesa aceleradamente en el ANE para traducir las características visuales a código LaTeX.

4. Redirección hacia la Vista de Resultados

Entrega de Resultados: `UniMERNetInferenceService` retorna la colección `List<LaTeXResult>`.

Navegación a Chat: `CropReviewViewModel` desactiva `isProcessing = false` e invoca `AppNavigationCoordinator.navigateToResults(results:)`. El coordinador

empuja el destino en [NavigationPath](#) y SwiftUI presenta la pantalla final con la interacción tipo chat y las opciones de copiado al portapapeles.

TX29

EquationSelectionView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz de selección de ecuaciones gestionando el estado de carga y restauración del lienzo. Dibuja las regiones trazadas previamente recuperadas desde caché y provee una interfaz de contingencia (*fallback*) para recargar el estado en caso de fallos en la lectura de persistencia.

Propiedades y métodos:

-**viewModel: EquationSelectionViewModel**: Instancia del ViewModel observada en SwiftUI para conmutar la visualización según el estado de la persistencia y los eventos del lienzo.

body: Some View: Jerarquía visual que renderiza el lienzo activo con las ROIs existentes o el botón de reintento según **viewModel.failedToLoadState**.

-**renderExistingROIs(): View**: Método privado que dibuja los rectángulos de las ROIs recuperadas del servicio de persistencia sobre la imagen base.

-**renderReloadFallbackButton(): View**: Método privado que renderiza la vista de error/reintento cuando falla la recuperación de la caché.

EquationSelectionViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con **@MainActor** y **@Observable** bajo Swift 6. Administra el ciclo de vida de carga del lienzo al aparecer la pantalla, orquesta la persistencia asíncrona de las ROIs en segundo plano y controla los estados de falla de lectura en memoria.

Propiedades y métodos:

equationROIs: List: Colección reactiva de regiones de interés actualmente activas en el lienzo.

failedToLoadState: Bool: Estado reactivo que indica si ocurrió un error durante la recuperación de las ROIs en caché.

isCanvasActive: Bool: Estado reactivo que habilita o deshabilita la capa de captura de gestos táctiles.

-persistenceService: ROIPersistenceServiceProtocol: Interfaz inyectada del servicio de persistencia de regiones.

loadStateOnAppear() async: Método invocado en el `.task` de la vista para solicitar las ROIs previamente guardadas al actor de persistencia.

addNewROI(normalizedRect: CGRect): Agrega una nueva ROI a la colección y dispara la persistencia asíncrona en segundo plano.

reloadSavedROIs() async: Método de reintento para volver a consultar la caché de ROIs si la carga inicial reportó error.

ROIPersistenceService (Capa de Servicios de Infraestructura / Actor):

Funcionalidad:

Servicio encapsulado como un `Actor` de Swift 6 para garantizar acceso concurrente seguro (*thread-safe*) al estado cacheado de las ROIs.

Persiste y recupera la colección de selecciones en memoria o almacenamiento diferido sin bloquear el hilo principal (`@MainActor`).

Propiedades y métodos:

-cachedROIs: List: Colección interna de ROIs retenidas en caché.

fetchCachedROIs() async -> List: Método asíncrono que consulta y retorna las ROIs almacenadas.

persistROIs(rois: List) async: Método asíncrono que actualiza la caché interna con el nuevo lote de ROIs trazadas.

EquationROI (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (**struct**) de la capa de dominio que abstrae una Región de Interés mediante su identificador único y sus coordenadas normalizadas.

Propiedades y métodos:

id: UUID: Identificador único inmutable para el diffing declarativo eficiente en colecciones de SwiftUI.

normalizedBounds: CGRect: Geometría delimitadora expresada en espacio de coordenadas normalizadas $\text{CGRect} \text{ in } [0.0, 1.0]$.

Flujo:

1. Carga e Inicialización de Estado al Aparecer la Vista

Acción de la Vista: Al cargarse **EquationSelectionView**, el modificador **.task** invoca asíncronamente **EquationSelectionViewModel.loadStateOnAppear()**.

Consulta al Actor: El ViewModel solicita la lista de selecciones llamando a **ROIPersistenceService.fetchCachedROIs()**.

Restauración de UI: El actor lee **cachedROIs** en un hilo secundario y devuelve las regiones. El ViewModel las asigna a **equationROIs**, establece **isCanvasActive = true** y SwiftUI renderiza las ROIs recuperadas con **-renderExistingROIs()**.

2. Registro y Persistencia Continua de Nuevas ROIs

Gesto de Trazado: El usuario dibuja una nueva región en el lienzo. Se llama a **EquationSelectionViewModel.addNewROI(normalizedRect:)**.

Actualización y Caché: El ViewModel crea la entidad **EquationROI**, la apila en **equationROIs** e invoca asíncronamente en segundo plano a **ROIPersistenceService.persistROIs(rois:)** para actualizar la caché sin congelar los 120 Hz de la interfaz.

3. Manejo de Fallos en Persistencia y Reintento (Fallback)

Ocurrencia de Error: Si la lectura inicial en **ROIPersistenceService** falla o lanza una excepción, el ViewModel captura el error y asigna **failedToLoadState = true**.

Vista de Contingencia: `EquationSelectionView` reacciona ocultando el lienzo activo y desplegando la interfaz de fallo mediante `-renderReloadFallbackButton()`.

Reintento Manual: El usuario presiona "Volver a cargar". La vista invoca `EquationSelectionViewModel.reloadSavedROIs()`, restableciendo `failedToLoadState = false` e iniciando nuevamente la consulta asíncrona al actor de persistencia.

TX30

LaTeXChatResultsView (Capa de Presentación SwiftUI):

Funcionalidad:

Renderiza la interfaz final de resultados interactivos con un diseño estilo feed/chat. Muestra cada recorte de ecuación (`CGImage`) alineado con su correspondiente código LaTeX generado por el pipeline de UniMERNet, proporcionando acciones rápidas para copiar la sintaxis al portapapeles de iOS, retornar a la selección de ROIs o reiniciar el flujo hacia la pantalla principal.

Propiedades y métodos:

`-viewModel: LaTeXChatResultsViewModel`: Instancia del ViewModel observada en SwiftUI para reaccionar a cambios en los estados de diálogos de confirmación y eventos de usuario.

`body: Some View`: Jerarquía visual adaptativa que renderiza la lista cronológica de resultados traducidos, la barra de acciones superior y las alertas emergentes de confirmación.

`-renderChatMessageList(): View`: Método privado que construye las tarjetas/burbujas individuales con la imagen recortada, el bloque de texto con sintaxis LaTeX y el botón de copiado rápido.

`-renderConfirmationAlerts(): View`: Método privado que renderiza las vistas modales emergentes para confirmar el regreso a la vista de selección o el reinicio total de la app hacia el menú inicial.

`-renderActionHeader(): View`: Método privado que dibuja la cabecera superior con las opciones globales "Volver a Selección" y "Nuevo Escaneo / Menú".

LaTeXChatResultsViewModel (Capa de Lógica de Presentación):

Funcionalidad:

Gestor de estado reactivo etiquetado con `@MainActor` y `@Observable` en Swift 6. Administra la colección de resultados traducidos (`LaTeXResultItem`), controla las banderas modales para prevenir la pérdida

accidental del historial de traducciones e interactúa con el servicio de portapapeles y el coordinador de navegación.

Propiedades y métodos:

resultItems: **List:** Colección reactiva de las ecuaciones procesadas y traducidas exitosamente.

showResetConfirmation: **Bool:** Estado reactivo que controla la visibilidad de la alerta modal para reiniciar la app al menú principal.

showReturnToSelectionConfirmation: **Bool:** Estado reactivo que controla la visibilidad de la alerta modal para regresar a la vista de selección de ecuaciones.

-coordinator: **AppNavigationCoordinator:** Referencia inyectada para gestionar las transiciones de rutas en la aplicación.

copyToClipboard(latex: String): Delega al servicio estático **ClipboardService** la copia de la cadena en sintaxis LaTeX al portapapeles del sistema.

requestResetToMainMenu(): Activa **showResetConfirmation = true** para desplegar el modal de confirmación antes de purgar la sesión.

confirmResetToMainMenu(): Cierra el modal e invoca **AppNavigationCoordinator.resetToInitialMenu()** para vaciar la pila de navegación.

cancelReset(): Desactiva **showResetConfirmation = false** dejando la pantalla intacta.

requestReturnToSelection(): Activa **showReturnToSelectionConfirmation = true** al solicitar el regreso a la pantalla de marcado.

confirmReturnToSelection(): Cierra el modal e invoca **AppNavigationCoordinator.popToSelectionView()**.

cancelReturnToSelection(): Desactiva **showReturnToSelectionConfirmation = false**.

LaTeXResultItem (Modelo de Dominio / Estructura):

Funcionalidad:

Estructura inmutable (`struct`) de la capa de dominio que almacena el resultado de la inferencia de una ecuación específica junto a sus metadatos de renderizado y tiempo.

Propiedades y métodos:

`id`: `UUID`: Identificador único inmutable para garantizar un diffing declarativo $O(1)$ en listas de SwiftUI.

`croppedImage`: `CGImage`: Referencia a la sub-imagen recortada y procesada.

`latexCode`: `String`: Cadena de texto resultante con la sintaxis LaTeX decodificada por el modelo UniMERNet.

`timestamp`: `Date`: Marca de tiempo de la generación del resultado.

ClipboardService (Capa de Servicios de Infraestructura):

Funcionalidad:

Servicio utilitario que encapsula la API nativa `UIPasteboard` de iOS para operaciones de copia de texto, manteniendo desvinculado el framework `UIKit` de la capa de vista.

Métodos y firmas:

`static copy(text: String)`: Método de clase que escribe el texto directo en `UIPasteboard.general.string`.

AppNavigationCoordinator (Capa de Arquitectura / Navegación):

Funcionalidad:

Coordinador centralizado de la pila declarativa `NavigationPath` responsable de conmutar los flujos de pantalla globales sin acoplar los ViewModels entre sí.

Propiedades y métodos:

`path: NavigationPath`: Instancia declarativa de la pila de navegación.

`resetToInitialMenu()`: Remueve todos los elementos de `path` restaurando la app al estado inicial del menú principal.

`popToSelectionView()`: Desapila las vistas necesarias hasta posicionar nuevamente al usuario en `EquationSelectionView`.

Flujo:

1. Renderizado de Resultados y Copiado al Portapapeles

Acción de Vista: Al cargar `LaTeXChatResultsView`, SwiftUI renderiza la lista de `resultItems` mediante `-renderChatMessageList()`.

Copiado: El usuario presiona "Copiar LaTeX" en una tarjeta específica. La vista invoca `LaTeXChatResultsViewModel.copyToClipboard(latex: item.latexCode)`. El ViewModel ejecuta `ClipboardService.copy(text:)` enviando la cadena a `UIPasteboard` y dispara un feedback háptico en iOS.

2. Flujo de Regreso a Selección (Agregar/Editar Ecuaciones)

Solicitud: El usuario presiona el botón "Volver a Selección".

Modal de Alerta: Se ejecuta `requestReturnToSelection()`, el ViewModel establece `showReturnToSelectionConfirmation = true` y SwiftUI renderiza `-renderConfirmationAlerts()`.

Confirmación: Al tocar "Confirmar", se ejecuta `confirmReturnToSelection()`, llamando a `AppNavigationCoordinator.popToSelectionView()`. El coordinador desapila las pantallas y retorna a `EquationSelectionView` conservando la imagen original.

3. Flujo de Reinicio Total (Nuevo Escaneo)

Solicitud: El usuario presiona "Nuevo Escaneo".

Modal de Alerta: Se ejecuta `requestResetToMainMenu()`, asignando `showResetConfirmation = true`.

Confirmación: El usuario presiona "Reiniciar". Se invoca `confirmResetToMainMenu()`, la cual llama a `AppNavigationCoordinator.resetToInitialMenu()`. La pila `NavigationPath` se vacía completamente y la aplicación regresa al menú de bienvenida inicial.